

LUMO: A Multi-agent AI Study Coach

Project Workbook

By

Alshama Mony Sheena
Bhavya Jain
Gautam Santhanu Thampy
Nivedita Nair

10/24/2025

Advisor: Bernardo Flores

TABLE OF CONTENTS

Chapter 1. Literature Search, State of the Art.....	3
1.1. Literature Search.....	3
1.1.1. Attention-Aware Learning and Engagement in Elementary Education.....	3
1.1.2. Mastery-Based Progression and Intelligent Task Ordering in Tutoring Systems.....	3
1.1.3. Existing AI-Based Tutoring Systems and Their Limitations.....	4
1.1.4. Towards Multi-Agent Architecture and their benefits.....	4
1.2. State-of-the-Art Summary.....	5
Chapter 2. Project Justification.....	8
Chapter 3. Project Requirements.....	9
3.1 Requirements.....	9
3.1.1 Functional Requirements.....	9
3.1.2 Non-Functional Requirements of Lumo.....	12
Chapter 4. Dependencies and Deliverables.....	14
4.1 Dependencies.....	14
4.2 Deliverables.....	15
Chapter 5. Project Architecture.....	17
5.1 Layers and components.....	17
5.2 Architecture Diagram.....	19
Chapter 6. Project Design.....	20
6.1. Design Goals and Constraints.....	20
6.2. Architecture Overview - Layers and Components.....	21
6.3. Component Contracts.....	29
6.3.1. Planner Contract.....	29
6.3.2. Lesson Designer Agent Contract.....	29
6.3.3. Quiz & Challenge Agent Contract.....	30
6.3.4. Feedback & Motivation Agent Contract.....	30
6.3.5. Analytics & Attention Agent Contract.....	30
6.4. Privacy Guardrails.....	31
6.4.1. Data Minimization and Consent.....	31
6.4.2. Data Retention and Anonymization.....	31
6.5. Security Controls.....	32
6.5.1. Role-Based Access Control and Authentication.....	32
6.5.2. Data Encryption and Audit Logging.....	33

Chapter 7. QA, Performance, Deployment Plan.....	34
7.1 QA Strategy and Traceability.....	34
7.2 Usability and Human-Centric Validation.....	36
7.2 LLM Output and Ethical Quality Assurance.....	37
7.2.1 Red-Team and Fallback Procedure.....	37
7.3 Performance Planning.....	38
7.3.1 Automated Evals for Agents.....	39
7.3.2 Advanced Agentic System Tests.....	40
7.4 Deployment Approach.....	41
7.5 Risk Awareness and Mitigation.....	42
7.6 Production Monitoring and Feedback.....	43
Chapter 8. Implementation Plan and Progress.....	44
8.1 Programming and Execution Environment.....	44
8.2 Development Tools for the Team Members.....	44
8.3 Implementation Flow.....	45
8.3.1 Phase 1: Environment and Contracts.....	45
8.3.2 Phase 2: Baseline Reproduction.....	45
8.3.3 Phase 3: Vertical Workflow for Lesson to Quiz.....	46
8.3.4 Phase 4: Closed Loop with Feedback and Attention Hooks.....	46
8.3.5 Phase 5: Analytics, Privacy and Transparency.....	46
8.3.6 Phase 6: Verification and Validation.....	46
Chapter 9. Project Schedule.....	47

Chapter 1. Literature Search, State of the Art

1.1. Literature Search

1.1.1. Attention-Aware Learning and Engagement in Elementary Education

Attention is vital for learning, along with time spent on a task and sustained focus in challenging topics. Research conducted in elementary level education highlights the importance of short focused lessons and instant feedback as producing strong impact on knowledge retention and mastery [1]. Also, significant contributions have been made by cognitive scientists in assessing and learning attention control in children. Bosch et al. [2] demonstrated that engagement can be quantified in real time through multimodal data in classroom settings, while Whitehill et al. [3] talks about the importance of facial expression analysis in understanding attention orientation. These techniques have measured how flaws in attention control can hinder effective absorption of knowledge, especially in younger children.

Engagement is also vital in adaptive education, and it has been studied using behavioural and multimodal signals, which serve as markers for sustained attention. Eye-gaze dynamics, for example, can identify mind wandering during reading as suggested by Bixler and D'Mello[4]. Existing facial expression recognition models are able to achieve human-level accuracy in identifying student engagement, which enables their use in real-time adaptive technologies [5]. The research conducted in these areas prove that attention and engagement are indeed vital measurable variables that can serve as inputs for intelligent tutoring systems.

1.1.2. Mastery-Based Progression and Intelligent Task Ordering in Tutoring Systems

Mastery-based learning means that the learners have to demonstrate proficiency before they can progress to the next level. Early research has shown that structured mastery based learning improves retention and transfer of knowledge by customizing the material to a student's evolving skill profile [6]. Therefore, knowledge tracing and Bayesian modeling approaches allow tutors to dynamically estimate mastery and assign practice. Piech et al. [7] used Deep Knowledge Tracing (DKT) applied recurrent neural networks to student mastery modeling, which saw improvements in predictive accuracy over traditional Bayesian frameworks. These show how computational models make use of mastery-based progression in adaptive learning environments.

Mastery-based curriculum has been researched in large-scale adaptive platforms. Brusilovsky and Millán [8] reviewed adaptive hypermedia and intelligent tutoring systems and found that ALEKS, a widely used mathematics system, has been shown to produce significant learning gains through individualized mastery-driven progression. Ritter et al. [9] reported significant learning gains from Cognitive Tutor/Carnegie Learning, demonstrating the impact of this method. Anderson et al. [10] proved that Khan

Academy enforces progression that depends on demonstrated competence, with systematic reviews confirming positive impacts on student achievement.

While mastery-based learning is important, intelligent task ordering extends upon the former by considering not only whether a learner has mastered a concept but also which next task will optimize growth. VanLehn [5] framed this distinction as the “inner loop” of step-level guidance and the “outer loop” of problem selection. Collectively, these findings say that mastery-based progression and intelligent sequencing are important in intelligent tutoring systems.

1.1.3. Existing AI-Based Tutoring Systems and Their Limitations

AI-based tutoring systems have now evolved from lab experiments to large-scale deployments. Cognitive Tutor/Carnegie Learning, ALEKS, and ASSISTments are some intelligent platforms that have proven more effective subjects like mathematics, when compared to traditional instruction and have produced gains equivalent to one standard deviation under experimental conditions [11]. These systems suggest that mastery-based progression and adaptive learning in classrooms using AI can go a long way in helping children. Despite the success of the above-mentioned systems, there are limitations and gaps in existing tools.

VanLehn [5] suggests that most of the existing tools rely heavily on the content delivery and assessment, rather than attention and engagement levels of the students. Nkambou et al. [12] elaborates on this by saying that while students benefit from individualized pacing, tutors often fail to sustain motivation, explain beyond correctness, or respond to small indicators of disengagement. Also commercial systems follow a closed-box architecture with no room for transparency in designing the curriculum from a parent/teacher perspective. These constraints highlight the need for next-generation approaches that extend beyond mastery tracking to incorporate engagement analytics, open architecture, and greater pedagogical flexibility.

1.1.4. Towards Multi-Agent Architecture and their benefits

The above mentioned limitations can be overcome by adopting multi-agent architecture in educative platforms. Unlike traditional tutoring systems, multi-agent designs distribute roles such as lesson design, feedback generation, evaluation, and analytics across specialized components that interact cooperatively. Nkambou et al. [12] suggest that modularization improves both scalability and transparency because each agent can be optimized for its pedagogical role, and system behavior becomes easier to explain to educators. Kim et al. [13] have done empirical studies in conversational tutoring which confirm that a combination of AI agents helps in more effective interaction and stronger learning.

Cognitive and motivational theories of learning also support multi-agent architecture. Graesser et al. [14] highlight that effective tutoring requires a “closed learning loop” where explanation, assessment, and motivation are continuously integrated, a principle that directly informs the design of multi-agent educational systems. For example, Murray

et al. [15] emphasize the importance of a system where specialized agents can monitor attention, provide adaptive feedback, and evaluate mastery continuously can be used in helping elementary kids to learn while also looping in teachers/parents. VanLehn [5] and Kim et al. [13] have proven that personalized scaffolding, affect-sensitive feedback, and adaptive curricula can be enabled by agent-based designs. Collectively the literature shows that multi-agent educational environments can mix the strengths of prior ITS with modular, role-based innovations therefore making them well-suited for building next-generation adaptive learning platforms for elementary education.

1.2. State-of-the-Art Summary

Recent advances in adaptive learning provide many leading-edge tools and techniques that move beyond static mastery-based tutoring toward adaptive, personalized and engagement-aware systems. At the architectural level, multi-agent frameworks have emerged as a powerful design strategy. This framework is different as it distributes roles such as lesson sequencing, feedback generation, and engagement monitoring across coordinated agents thus improving transparency and scalability. Empirical studies show that multi-agent conversational tutors increase both the richness of interactions, and student learning outcomes [12][13].

Large language models (LLMs) now power tutoring systems that generate contextualized explanations and adaptive practice questions. LLM-based automatic question generation produces problem sets with configurable difficulty and curriculum alignment, enabling individualized practice that adapts to learner progress [17]. Similarly, prompting methods such as chain-of-thought help with step-by-step reasoning that leads to targeted hints and explanations, offering more than correctness-only feedback [18]. These LLM tools help in designing intelligent systems with real-time adaptiveness.

Engagement analytics is another critical tool. Multimodal signals are being used as indicators that can identify whether a student is focused and attentive. Whitehill et al. [3] showed that facial expression recognition can capture engagement with accuracy comparable to human observers whereas Bosch et al. [2] demonstrated that engagement can be measured in real time through multimodal data in classroom context. Keystroke and gaze-based features further serve as indicators of unattentiveness of a student [4]. These methods extend adaptive tutors with the capability to react to moment-to-moment changes in learner attention, a feature particularly important in elementary education where focus is easily lost.

In a nutshell, the state of the art in educational AI is characterized by multi-agent architectures, LLM-driven personalization and real-time engagement analytics using multimodal signals. These tools and techniques represent the most advanced capabilities available today in adaptive learning research and the design of next-generation systems for elementary learners.

1.3. References

- [1] A. Buchem and R. Hamelmann, “The effects of microlearning: A scoping review,” *Educational Technology Research and Development*, vol. 70, pp. 211–240, 2022, doi: 10.1007/s11423-022-10084-1.
- [2] N. Bosch, S. D’Mello, R. Baker, J. Ocumpaugh, and V. Shute, “Temporal generalizability of face-based engagement detection,” in *Artificial Intelligence in Education (AIED 2016)*, Lecture Notes in Computer Science, vol. 9684, pp. 38–49, Springer, 2016.
- [3] J. Whitehill, Z. Serpell, Y.-C. Lin, A. Foster, and J. R. Movellan, “The faces of engagement: Automatic recognition of student engagement from facial expressions,” *IEEE Transactions on Affective Computing*, vol. 5, no. 1, pp. 86–98, Jan.–Mar. 2014.
- [4] R. Bixler and S. K. D’Mello, “Automatic gaze-based user-independent detection of mind wandering during computerized reading,” *User Modeling and User-Adapted Interaction*, vol. 26, no. 1, pp. 33–68, Mar. 2016.
- [5] K. VanLehn, “The behavior of tutoring systems,” in *Artificial Intelligence in Education: AI-ED 2007*, Lecture Notes in Computer Science, vol. 4578, pp. 668–675, Springer, 2007.
- [6] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, Dec. 1995.
- [7] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Proceedings of the ACM Conference on Learning at Scale (L@S ’15)*, ACM, 2015, pp. 1–10.
- [8] P. Brusilovsky and E. Millán, “User models for adaptive hypermedia and adaptive educational systems,” in *The Adaptive Web*, Lecture Notes in Computer Science, vol. 4321, pp. 3–53, Springer, 2007.
- [9] K. R. Koedinger, J. R. Anderson, W. H. Hadley, and M. A. Mark, “Intelligent tutoring goes to school in the big city,” in *Proceedings of the 8th World Conference on Artificial Intelligence in Education (AIED)*, Springer, 1997, pp. 421–428.
- [10] R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, “Cognitive tutors: Lessons learned,” in *Proceedings of the International Conference on Artificial Intelligence in Education (AIED 1995)*, Springer, 1995, pp. 1–19.
- [11] K. R. Koedinger, A. T. Corbett, and C. Perfetti, “The role of cognitive tutors in supporting learning: Problem solving and skill acquisition,” in *Technology Enhanced*

Learning: Design Patterns and Pattern Languages, Lecture Notes in Computer Science, vol. 4753, Springer, 2008, pp. 140–165.

[12] R. Nkambou, J. Bourdeau, and R. Mizoguchi, *Advances in Intelligent Tutoring Systems*, Studies in Computational Intelligence, vol. 308, Springer, 2010.

[13] J. Kim, E. Arroyo, K. E. Boyer, and W. L. Johnson, “Multiple agent designs in conversational intelligent tutoring systems,” *Technology, Knowledge and Learning*, vol. 24, no. 4, pp. 569–592, 2019.

[14] A. C. Graesser, S. Lu, G. Jackson, H. Mitchell, M. Ventura, A. Olney, and M. Louwerse, “AutoTutor: A tutor with dialogue in natural language,” *Behavior Research Methods, Instruments, & Computers*, vol. 36, no. 2, pp. 180–192, May 2004. (Springer)

[15] T. Murray, S. Blessing, and S. Ainsworth, *Authoring Tools for Advanced Technology Learning Environments: Toward Cost-Effective Adaptive, Interactive, and Intelligent Educational Software*, Springer, 2003.

[16] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *International Journal of Artificial Intelligence in Education*, vol. 30, pp. 121–204, Springer, 2020.

[17] I. L. Molina, V. Švábenský, T. Minematsu, L. Chen, F. Okubo, and A. Shimada, “Comparison of Large Language Models for Generating Contextually Relevant Questions” in *Technology Enhanced Learning for Inclusive and Equitable Quality Education (EC-TEL 2024)*, Lecture Notes in Computer Science, vol. 15160. Cham, Switzerland: Springer, 2024, pp. 137–143. doi: 10.1007/978-3-031-72312-4_18.

[18] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Chapter 2. Project Justification

Traditional teaching practices often conform to a "one-size-fits-all" approach, which has a propensity to exclude slower learners while insufficiently challenging more capable students. These flaws cause learners to disengage, gain superficial insights, and yield variable results. Attempts to redress this balance by means of adaptive educational technologies have arisen; however, many are still focused on distributing static content or checking the correctness rather than aiming to address the full cycle including lesson design, practice, assessment, and learner engagement. There is a strong need for systems that can replicate and extend the roles played by educators, tutors, and assessors on a scalable, data-informed basis.

This requirement is met through the suggested LUMO system, which organizes instruction into short, developmentally appropriate lessons, followed then by immediate assessment through quizzes. Concepts are broken down into manageable chunks, so as to alleviate cognitive overload as well as allow preparedness for mastery assessment. A Lesson Designer Agent introduces each concept through contextualized examples that correspond to the learner's personal pace as well as attentional traits. Through the use of misconception-based challenges within quizzes, the Quiz Agent ensures that children cannot simply guess their answers, thereby making mastery measured rather than superficial. The Attention Agent automatically tracks attentional rhythms to recognize the moments the learner is best attuned to receive the next piece of content. It identifies drift or overload, suggests breaks at the appropriate time, and schedules more challenging topics optimally, once attention is most concentrated.

Educators and parents receive a dashboard to monitor not just the content learned, but the dynamics of attention during periods of studying. This degree of awareness takes personalization beyond the content, to the "when" and the "how" of studying. It completes the learning cycle, as well, turning errors into constructive learning experiences. It corrects wrong answers with explanatory clarifiers, suggestive reminders, and motivating prompts to integrate assessment into the teachable process. It, thereby, reinforces the intellectual as well as the emotional aspect of instruction, engendering persistence, long-range proficiency, as well as building confidence.

Together, these innovations validate the project by addressing broadly acknowledged shortcomings within existing tutoring platforms, such as a lack of adequate real-time engagement awareness, inadequate transparency to the instructor, as well as too great a reliance upon accuracy-centered advancement. LUMO extends the practice of educational technology by bringing together attention-aware scheduling, adaptive evaluation, personalized lesson planning, and emotionally nurturing response within a single multi-agent framework. It fulfills the need to provide scalable, personalized, and morally aligned educational assistance for primary schooling.

Chapter 3. Project Requirements

3.1 Requirements

3.1.1 Functional Requirements

Table 1. List of requirements

Requirement	Priority	User Story	User Purpose	Acceptance Criteria
Lesson Designer – Core Lesson	Essential	When a student starts a new topic, the agent generates a short, curriculum-aligned lesson with visuals/stories.	To receive an engaging introduction to a concept.	Lesson generated quickly; aligned to curriculum; includes at least one story/visual/example.
Lesson Designer – Contextual Explanation	Essential	When a student asks “why” or “how,” the agent provides a contextual explanation in simpler words.	To clarify confusing points.	Agent responds to “why/how” prompt with a clear, age-appropriate explanation; response logged in dashboard.
Lesson Designer – Multi-Format Delivery	Desired	Lessons can be received in multiple formats: text, audio narration, drag-and-drop activity.	To support varied learning styles.	All formats available; students can toggle mode; no content mismatch between formats.
Quiz Agent – Adaptive Difficulty	Essential	After a lesson, student receives a quiz that adjusts difficulty in real time based on performance.	To practice at the right level of challenge.	Quiz difficulty rises if $\geq 80\%$ correct, drops if $\leq 40\%$; difficulty level logged for analysis.

Quiz Agent – Trap-Based Challenge	Desired	For common misconceptions, quiz inserts trap-based distractors (wrong but plausible answers).	To expose and correct misconceptions.	Distractor chosen from error taxonomy; if selected, system triggers Feedback Agent explanation.
Quiz Agent – Summary Chain	Essential	At the end of a unit, user/student receives a summary-chain quiz covering core steps in order.	To reinforce understanding of the whole process.	At least one summary chain per unit; 80% of steps answered correctly before moving ahead.
Quiz Agent – Gamified Mini-Game	Desired	Some quizzes appear as quick mini-games (timed, puzzle format).	To keep assessment fun and motivating.	Game runs smoothly; scoring ties back to mastery tracking; fun mode toggle available.
Feedback Agent – Immediate Correction	Essential	After a wrong answer, the system instantly shows why the answer is incorrect.	To fix errors quickly.	cites correct reasoning; positive tone confirmed by sentiment check.
Feedback Agent – Hint Ladder	Essential	Student can request hints that go from a light cue → guided step → worked example.	To scaffold learning step by step.	At least one hint always available; max 3 hints; hints lead to correct solution in ≤ 2 retries.
Feedback Agent – Motivation & Nudges	Essential	After multiple errors, the agent sends encouragement (“You’re close!”) or nudges (“Try one more before a break”).	To maintain persistence and discipline.	Encouragement always positive; no punitive tone; logs show reduced quit rate after nudges.

Feedback Agent – Re-Quiz After Support	Essential	After hints or reinforcement (e.g., mini-video), student is re-quizzed on the same concept.	To verify mastery after correction.	Re-quiz delivered automatically; mastery rate $\geq 80\%$ after re-quiz; progress updates only after success.
Attention Agent – Peak Time Scheduling	Essential	Tracks daily/weekly rhythms; suggests hard topics in high-focus windows.	To learn difficult material at optimal times.	At least 1 peak window logged per day; hard tasks assigned in the window; dashboard shows “focus time” heatmap.
Attention Agent – Drift Detection	Essential	Monitors latency/errors; detects drifting focus and suggests micro-recap or lighter task.	To keep student engaged when losing focus.	Drift flagged within 1 min; recap task triggered; $\geq 50\%$ of students re-engage (measured by accuracy uplift).
Attention Agent – Break Suggestion	Essential	If overload is detected (error streak/slow times), the agent suggests 1–2 min break with a resume card.	To prevent fatigue and frustration.	Break suggestion per day; resume card accurate;
Attention Agent – Mastery Tracking	Essential	Updates mastery model continuously; links attention patterns to concept learning.	To give teachers/parents deeper insight into progress.	Mastery scores update in the dashboard in real time; report shows attention vs. mastery correlation.
Parent/Teacher Dashboard – Progress	Essential	Parents/teachers log in to view mastery, time spent, strengths/weaknesses.	To monitor learner progress.	Dashboard with accuracy %, time per concept, and strengths/weaknesses clearly.

Adaptive Scheduling	Desired	Lessons auto-rescheduled to align with detected peak focus periods.	To optimize study sessions.	Lesson schedule updates visible; parents/students confirm reordering in the dashboard.
Gamified Micro-Interventions	Desired	Warm-up mini-games before lessons to boost attention.	To prepare learners mentally for new topics.	Game launches successfully
Self-Report Mechanism	Desired	The learner answers quick pings about focus/energy to calibrate the model.	To improve accuracy of attention modeling.	Self-report integrated; correlation visible between self-report and detected attention.
Advanced Attention Modeling	Optional	Opt-in use of camera/microphone for richer signals (processed locally, no raw storage).	To improve detection precision.	Explicit opt-in required;
Federated Learning	Optional	Device models share anonymized updates, not raw data.	To personalize while protecting privacy.	Federated updates run without leaks;

3.1.2 Non-Functional Requirements of Lumo

The non-functional requirements safeguard quality, usability and trust in the system.

1. Privacy and Security: The system ensures that no raw audio or video data is ever stored, protecting children’s sensitive information. This requirement is critical because the platform will be used by minors, and maintaining confidentiality builds trust with families.

2. Transparency: Every adaptive action taken by the system is accompanied by a clear explanation. For example, if a hint is suggested, the system explains that it is “because a similar mistake was made earlier.” This feature is required to help both learners and

parents understand the reasoning behind system decisions, increasing confidence and acceptance.

3. Responsiveness: Lessons, quizzes, and feedback must respond quickly to student interactions. Delays break the flow of learning and risk disengagement, so ensuring near-real-time system responses is essential for maintaining engagement.

4. Scalability: The platform should handle many learners and diverse curriculum standards simultaneously. This ensures that the system can grow to accommodate classrooms and schools of varying sizes without performance issues.

5. Usability: The learner interface must remain simple, child-friendly, and engaging, while the parent/teacher dashboard must be intuitive and actionable. This dual design requirement ensures accessibility for both young students and adult supervisors.

Chapter 4. Dependencies and Deliverables

4.1 Dependencies

Table 2. List of dependencies

Dependency	Purpose	Risks / Concerns	Mitigation / Alternatives
On-device analytics & telemetry	Collect and process real-time interaction data (latency, idle time, error rates) for attention modeling	Privacy concerns, device performance issues, potential rejection if using 3rd-party SDKs	Build lightweight in-house analytics; process locally; provide “minimal mode” with limited tracking
Secure storage & user authentication	Protect sensitive student/parent data, manage roles, and enforce access control	Vendor lock-in (e.g., Auth0, Cognito), compliance risks (COPPA, FERPA, GDPR)	Use standards (OAuth2, OIDC); support multiple IdPs; encrypt at rest & in transit; strong consent logs
Curriculum-aligned repositories & educational content	Provide lessons aligned with standards, including licensed images/videos	Paywalls, license restrictions, inconsistent quality	Prefer Open Educational Resources (OER) repositories (CK-12, OpenStax); track licenses; build curated in-house content library
Text-to-Speech (TTS) and visual rendering APIs	Enable multi-modal delivery of lessons (audio narration, visuals)	Pay-per-use APIs can increase costs; latency may disrupt flow	Use on-device TTS where possible; cloud TTS as premium; cache frequently used voices and visuals

Question banks & assessment engine	Deliver validated quizzes with adaptive difficulty and trap-based distractors	Gaps in coverage, reliance on external Open Educational Resources (OER) quality	Combine Open Educational Resources (OER) with educator-authored questions; continuous validation; fallback to fixed difficulty tiers
Feedback & Motivation (LLM integration)	Provide instant hints, motivational nudges, and re-quiz generation	High cost and latency for cloud LLMs; risk of age-inappropriate outputs	Cache scripted hints; small on-device models for common cases; human-in-the-loop calibration
Attention Agent (optional sensors: camera/mic)	Capture richer attention signals (eye gaze, voice activity) for modeling	Major privacy risks, ethical concerns, parent/school resistance	Strictly opt-in; on-device processing only; no raw data storage; always provide a “no-sensor” fallback
Ethical approval & compliance	Enable pilots with minors in schools	Legal/ethical barriers; delays in research approval	Prepare IRB/ethics applications; parental consent workflows; align with child-safety regulations

4.2 Deliverables

The project will produce three primary deliverables.

1. Multi-Agent Learning Application: A functional application integrating four orchestrated agents, the Lesson Designer, Quiz and Challenge, Feedback and Motivation, and Analytics and Attention agents. The tool delivers an adaptive, interactive learning experience for elementary school students. This prototype demonstrates curriculum-aligned lesson generation, adaptive assessments, immediate feedback, and attention modeling.

2. Parent/Teacher Dashboard: A dedicated dashboard agent that consolidates student mastery, time spent, attention patterns, and identifies strengths and weaknesses. This tool

equips parents and teachers with evidence-based insights to better support learners and manage schedules effectively.

3. Documentation Report: A comprehensive written report detailing the system design choices, implementation approach, performance evaluation, user feedback, and proposed future enhancements. This document serves as both a technical reference and a roadmap for scaling the system further.

Chapter 5. Project Architecture

The LUMO platform will adopt a multi-agent, layered, service-oriented architecture coordinated by a Planner/Orchestrator that routes learning goals (e.g., *teach fractions*, *quiz addition*) to specialized agents. Functionally, the system closes the learning loop by sequencing micro-lessons, adaptive quizzes, tiered feedback, and attention-aware scheduling, while simultaneously sharing progress and attention trends to parents and teachers through a dashboard. This design addresses the core requirement to show both the functional roles and the technologies we will be using.

5.1 Layers and components

The UI layer (web/mobile) would be implemented with Next.js/React/TypeScript, providing accessible lesson delivery, brief activities, and quiz/game interactions, plus a dedicated Parent/Teacher Dashboard.

The API & Orchestration layer (FastAPI) would handle authentication, sessions, and the central Planner, which would decompose goals and delegate work to agents.

The Agent layer will have four cooperating services: Lesson Designer (renders short, contextual lessons), Quiz & Challenge (adaptive items with misconception-focused distractors), Feedback & Motivation (encouraging nudges, Socratic hints, re-quiz triggers), and Analytics & Attention (continuous mastery and attention modeling).

Shared services include a User Model (mastery and focus profile), a Tool Pool (quiz bank, mini-games, and guarded LLM utilities), and a Policy/Ethics module for guardrails and transparency messaging.

Data & Storage centers on Postgres with content, events, and learner schemas, plus telemetry/analytics logs. Redis/Celery will support async workloads where latency would interrupt flow.

Runtime flow¹: A typical session begins when a student selects a topic. The Planner requests a concise lesson from the Lesson Designer; once completed, the Quiz Agent serves items at calibrated difficulty with targeted distractors. The Feedback Agent intervenes on errors with tiered hints and can trigger focused re-quizzing, turning mistakes into teachable moments. Throughout, the Analytics & Attention Agent updates

¹ TypeScript, *JavaScript with syntax for types*, available at: <https://www.typescriptlang.org/>

Next.js, *React Framework for Production*, available at: <https://nextjs.org/>

React, *A JavaScript library for building user interfaces*, available at: <https://react.dev/>

FastAPI, *Modern, fast (high-performance) web framework for building APIs with Python*, available at: <https://fastapi.tiangolo.com/>

PostgreSQL, *Open Source Relational Database*, available at: <https://www.postgresql.org/>

Redis, *In-memory database and cache*, available at: <https://redis.io/>

Celery, *Distributed task queue*, available at: <https://docs.celeryq.dev/>

mastery and attention state from interaction signals (e.g., response times and error patterns) and advises the Planner to continue, remediate, or suggest a short break; the Dashboard receives live updates on progress and attention dynamics for adults.

5.2 Architecture Diagram

For reference, the actual diagram can be found at this [link](#).

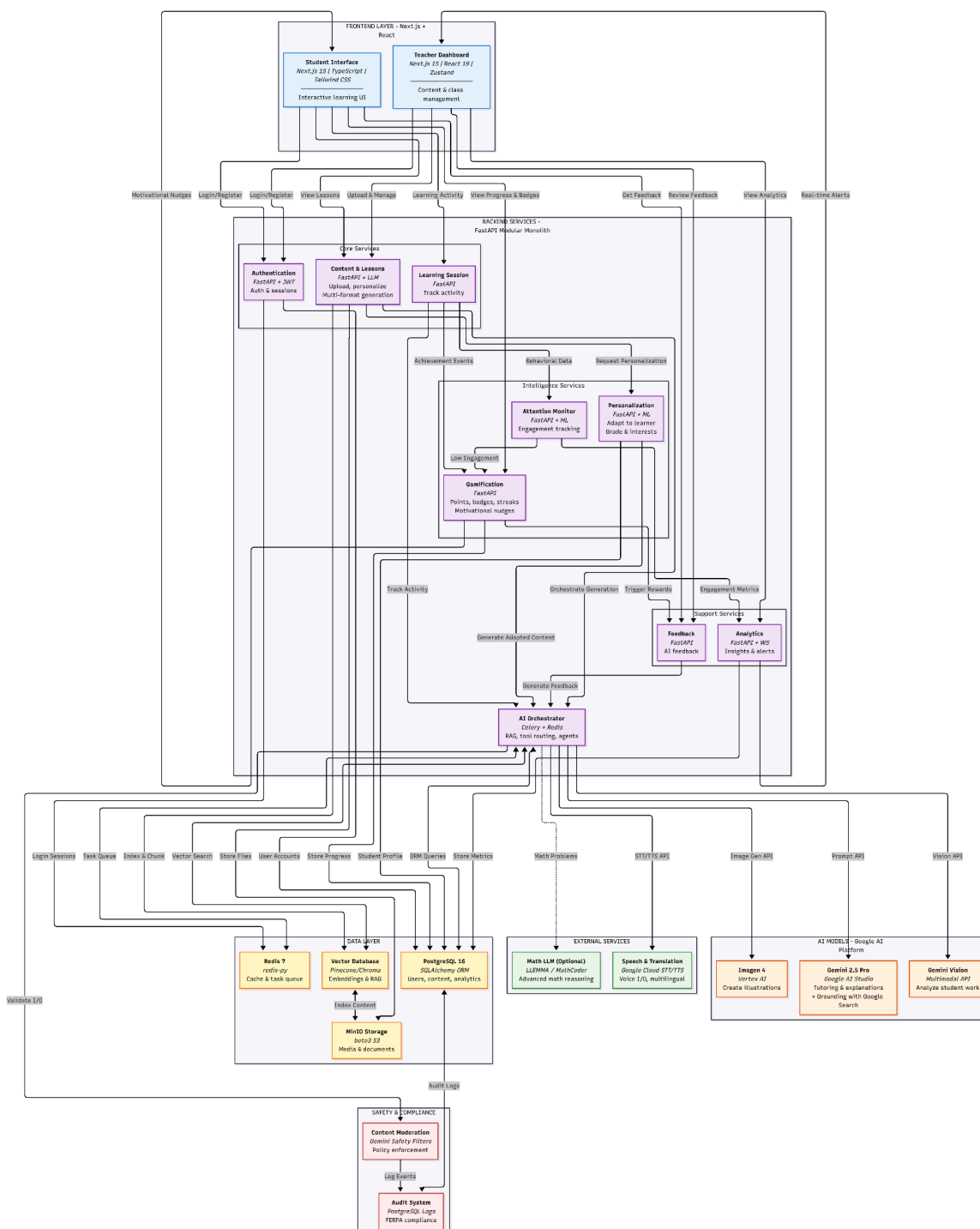


Figure 1. Proposed architecture diagram

Chapter 6. Project Design

6.1. Design Goals and Constraints

6.1.1. Design Goals

The LUMO platform aims to deliver a personalized and adaptive learning experience through the following key goals:

1. **Micro-lessons:** Deliver short, curriculum-aligned lessons tailored to each student's grade and proficiency level. (Managed by Lesson Designer Agent)
2. **Adaptive Quiz:** Provide quizzes that dynamically adjust difficulty based on student performance, reinforcing mastery and identifying misconceptions. (Managed by Quiz & Challenge Agent)
3. **Hints:** Offer tiered, Socratic hints to scaffold learning when students struggle, helping them understand errors without frustration. (Managed by Feedback & Motivation Agent)
4. **Re-quiz:** Enable re-quizzing on concepts after hints or motivational nudges to verify mastery before progression. (Managed by Feedback & Quiz Agents)
5. **Attention-aware Pacing:** Monitor learner attention and engagement to suggest breaks, micro-recaps, or adjust content pacing for optimal focus. (Managed by Analytics & Attention Agent)
6. **Adult Dashboards:** Provide parents and teachers with clear dashboards showing student mastery, attention trends, strengths, and areas needing support. (Managed by Analytics & Attention Agent)

6.1.2. Constraints

The system is designed with the following essential constraints to ensure ethical, secure, and usable operation:

1. **Child Privacy:** No storage of raw audio or video data to protect sensitive information of minors.
2. **Transparency:** Every adaptive action includes an explanatory "because..." message to improve learner and adult understanding of system decisions.
3. **Responsiveness:** Low-latency interactions to maintain engagement and seamless learning flow.

4. **Scalability:** Architecture supports growth in number of users and content without degradation of performance.
5. **Usability:** Interfaces must be child-friendly, intuitive, and accessible for young learners and easy to navigate for parents and teachers.

6.2. Architecture Overview - Layers and Components

The LUMO platform is structured into distinct layers, each responsible for a specific part of the system's overall functionality. This layered approach promotes modularity, scalability, and clear separation of concerns, enabling the development team to build, maintain, and evolve each part independently while ensuring smooth integration across the platform.

6.2.1. Frontend Layer (User Interface)

The frontend layer provides the primary interaction point for users, encompassing both students and adult supervisors such as parents and teachers. Implemented using Next.js and TypeScript, this layer delivers a responsive and accessible web application that supports micro-lessons, adaptive quizzes, tiered feedback, and dashboards. Lessons are rendered using MDX to combine rich text with interactive content. The frontend communicates exclusively with the backend through the Planner API to ensure consistent orchestration of user requests and system responses. Emphasis on usability and accessibility (including compliance with WCAG guidelines) ensures the platform is engaging and easy to navigate for all users, especially elementary-aged students.

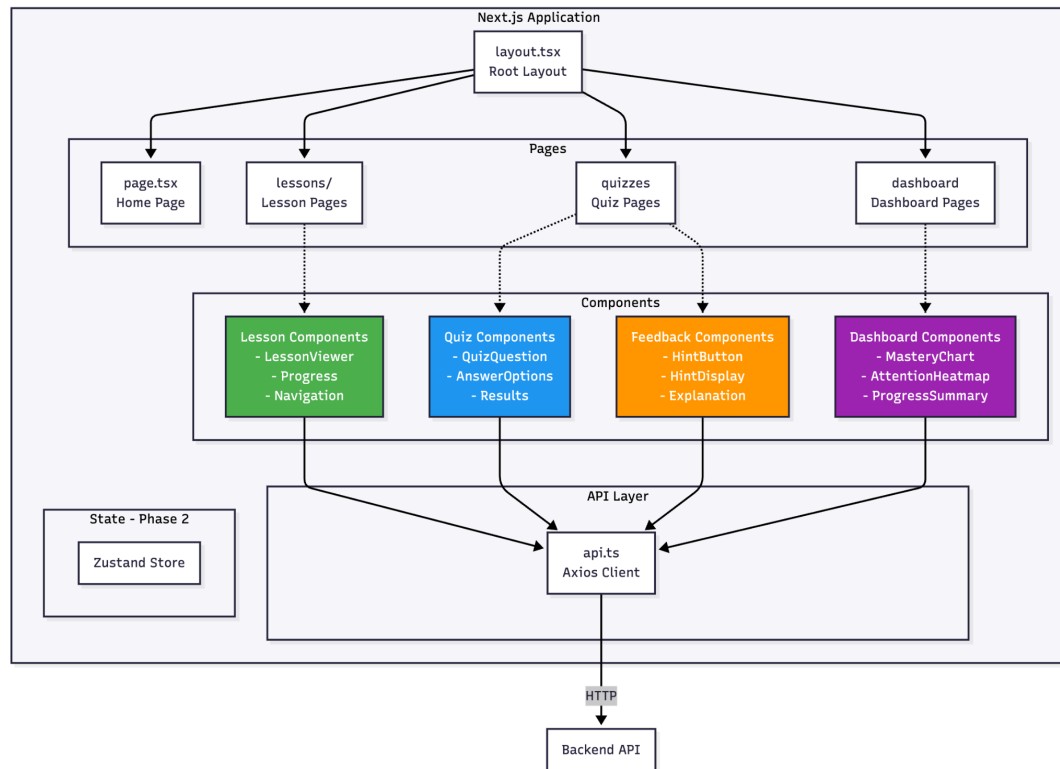


Figure 2. Frontend Component Structure

6.2.2. API and Orchestration Layer (Planner)

At the heart of the system lies the Planner, implemented as a FastAPI service. The Planner orchestrates all learning workflows by decomposing high-level user intents into concrete tasks delegated to specialized agents. It manages session states, sequences lessons and quizzes, and aggregates results and rationales to deliver coherent and transparent user experiences. Acting as the central coordinator, the Planner ensures that all adaptive decisions — such as difficulty adjustments, hint provision, and attention-based pacing — are executed consistently and efficiently. It also enforces security, privacy, and rate limiting policies at the API level.

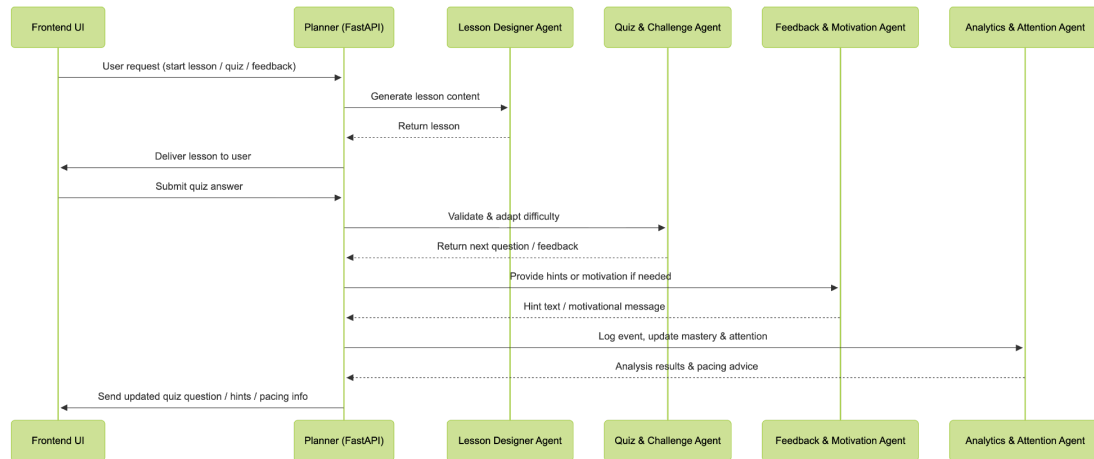


Figure 3. Sequence Diagram of Planner and Learning Agent Interactions

6.2.3. Agent Layer

The agent layer consists of four cooperating microservices, each encapsulating a specific pedagogical function:

- Lesson Designer Agent:** Responsible for generating curriculum-aligned micro-lessons tailored to the learner's profile. It fetches tagged content and media assets and assembles lessons that include context-sensitive explanations and illustrative examples.

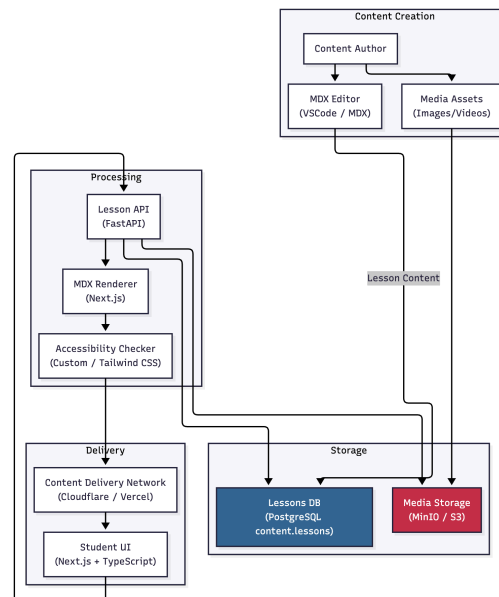


Figure 4. Lesson Designer Agent Workflow

- Quiz and Challenge Agent:** Creates adaptive quizzes that adjust question difficulty based on real-time learner performance. It integrates with an LLM provider to generate plausible distractors targeting common misconceptions, thereby enhancing mastery assessment.

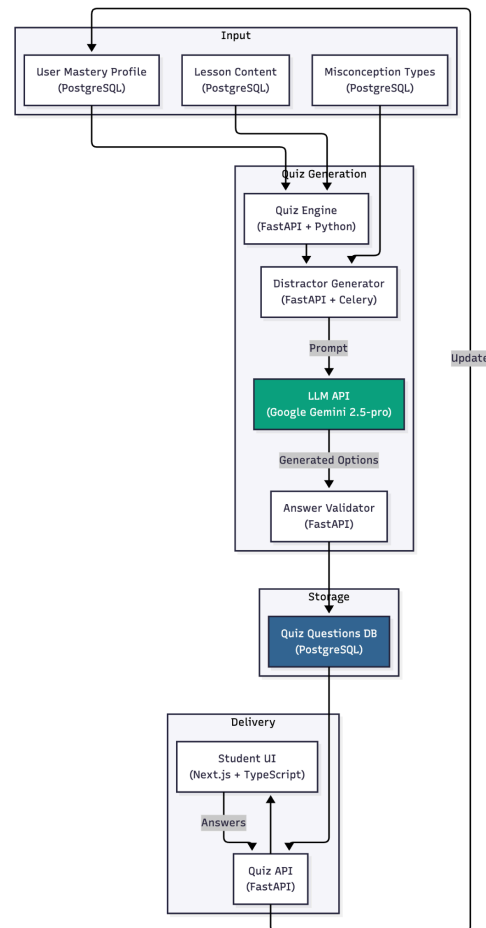


Figure 5. Quiz Generator Agent Workflow

- Feedback and Motivation Agent:** Provides tiered hints and motivational prompts to support learners when they struggle. It leverages both predefined templates and LLM-generated content, ensuring scaffolding is appropriate, timely, and encouraging.

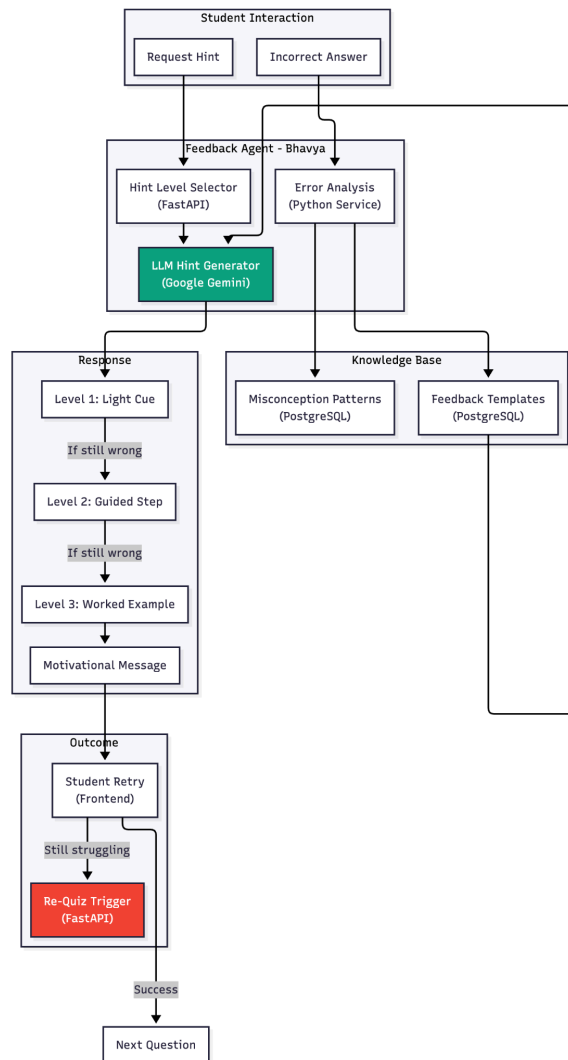


Figure 6. Feedback Agent Flow

- Analytics and Attention Agent:** Continuously monitors learner interactions to update mastery models and detect attention drift. It advises the Planner on pacing adjustments, break suggestions, and provides aggregated data for the adult dashboards. Each agent operates independently but coordinates through the Planner to ensure a seamless learning loop.

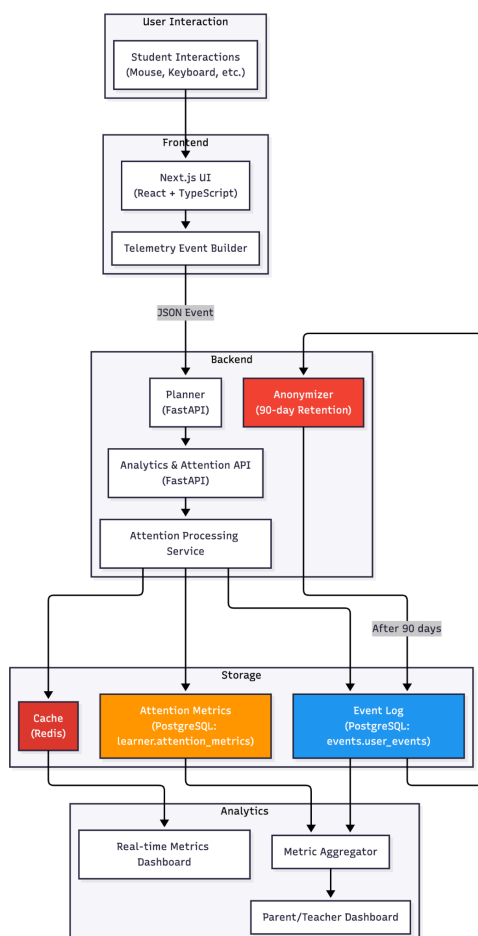


Figure 7. Attention Agent Data Flow

6.2.4. Data and Storage Layer

The data layer manages persistent storage and caching to support real-time performance and durability requirements. PostgreSQL serves as the primary data store for content, user profiles, learner mastery scores, event logs, and analytics data. Redis is used for caching frequently accessed data and managing asynchronous tasks to improve responsiveness. Media assets such as images and videos associated with lessons are stored in MinIO, an object storage service compatible with S3 APIs. The data layer implements strict access controls and follows privacy guardrails, including data retention policies and anonymization after 90 days.

6.2.4.1. Database Entity-Relationship Diagram (ERD)

This diagram illustrates the relational database schema underlying LUMO's data storage. It details the main tables across three logical schemas — content, events, and learner — and their relationships. Key entities include lessons, quiz questions, feedback templates,

user events, sessions, user profiles, mastery scores, and attention metrics. The ERD provides a foundational view of how educational content, learner interactions, and analytic metrics are persistently organized and linked to support adaptive learning.

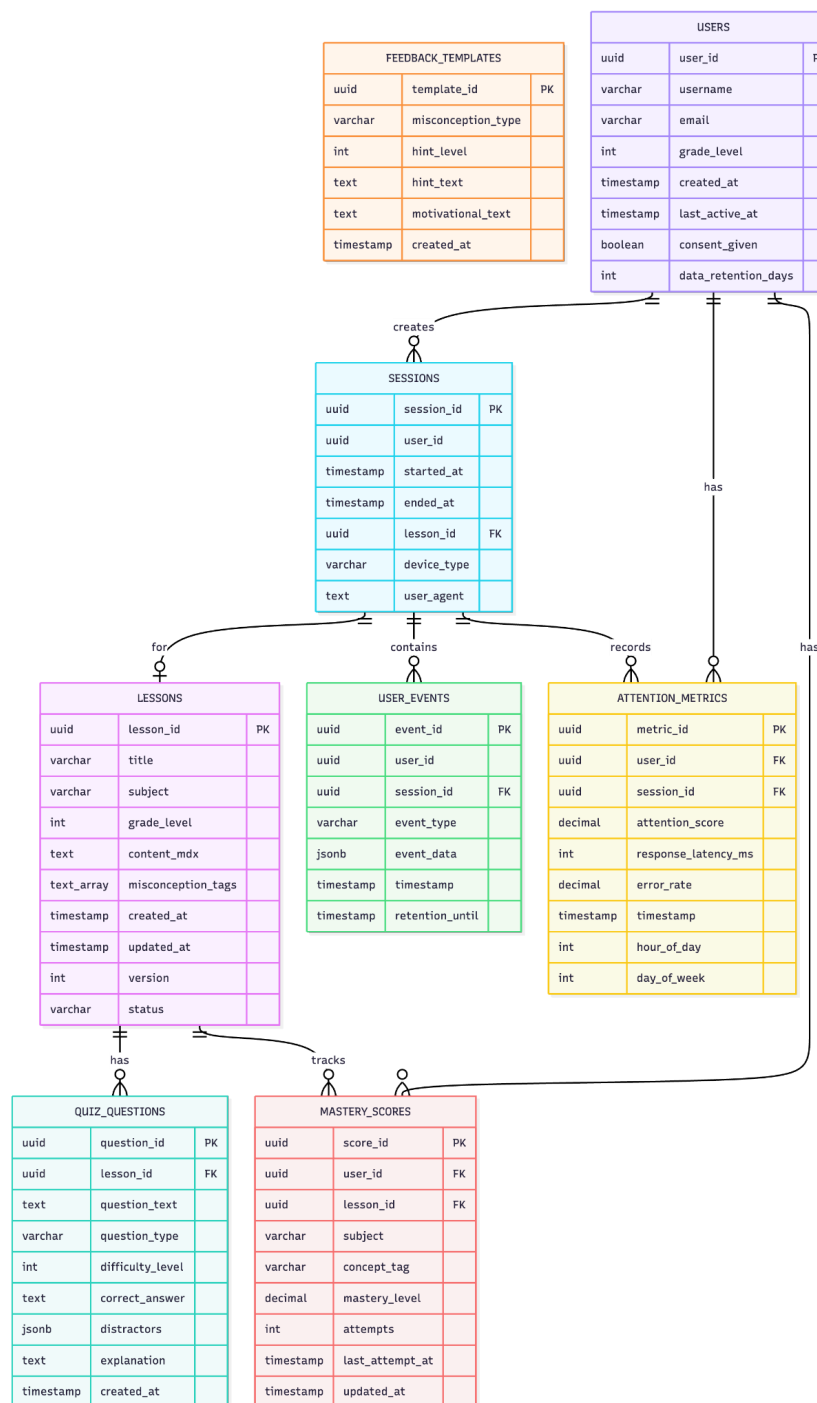


Figure 8. Entity-Relationship Diagram showing LUMO's database schema including content, event logging, and learner profiles.

6.2.4.2. Schema Organization Diagram

This diagram presents a high-level overview of the logical grouping of database schemas within LUMO. It separates data into **content** (lessons, quizzes, feedback), **events** (user interaction logs and sessions), and **learner** (user profiles, mastery, attention metrics) schemas. Arrows indicate the references and dependencies between these schemas, demonstrating how data flows and integrates across the platform's persistent storage.

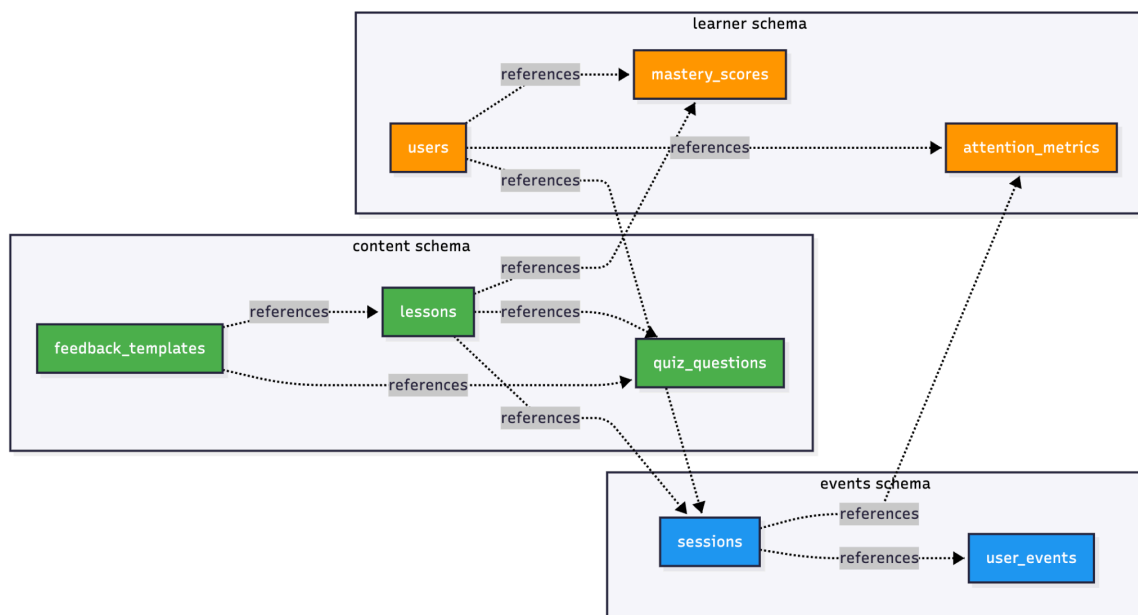


Figure 9. Schema Organization Diagram depicting logical grouping and relationships among LUMO's content, events, and learner data schemas.

6.2.5. External Services Layer

LUMO integrates with external services to enhance functionality and scalability. The most critical integration is with a Large Language Model (LLM) provider, such as Google Gemini, which supports dynamic generation of distractors, hints, and explanations with appropriate safety filters. An identity provider may also be used for authentication and role-based access control, ensuring secure user sessions. All external interactions are managed carefully to preserve user privacy and maintain system performance.

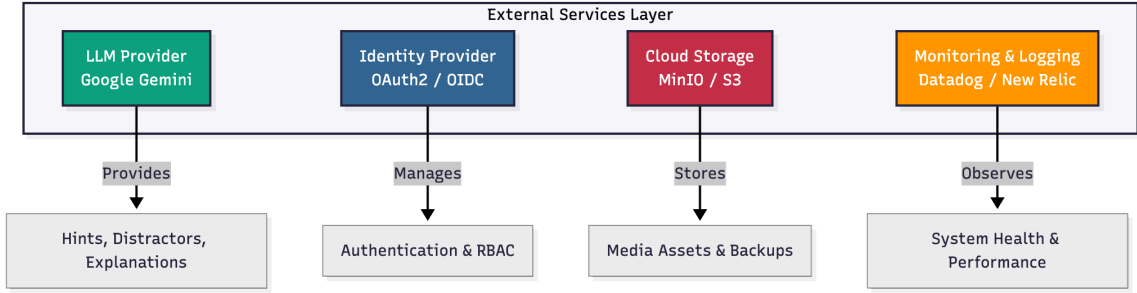


Figure 10. External Services Layer

6.3. Component Contracts

In this section, we specify the formal contracts for each core component of the LUMO system, defining their inputs, outputs, operational preconditions and postconditions, dependencies, service level agreements (SLAs), and failure handling strategies. These contracts provide clear technical boundaries and expectations, ensuring consistent behavior across the platform and guiding implementation and testing.

6.3.1. Planner Contract

Inputs: The Planner receives high-level user requests, such as starting a lesson, submitting quiz answers, or requesting hints, along with session and learner context.

Outputs: It produces sequenced learning actions—lessons, quiz questions, hints, motivational prompts—and transparency rationales explaining adaptive decisions.

Preconditions: An active user session and loaded learner profile are required before processing.

Postconditions: The Planner dispatches appropriate tasks to agents, updates session state, and persists mastery data.

Dependencies: Coordination with all agents (Lesson Designer, Quiz, Feedback, Analytics), backend databases, and external LLM services via agents.

SLA: Most requests are processed within one second.

Failure Handling: If an agent is unavailable, the Planner falls back to cached content or default sequences, logging issues while maintaining user flow.

6.3.2. Lesson Designer Agent Contract

Inputs: Requests specifying the learner’s current topic, grade level, and contextual profile to tailor content appropriately.

Outputs: High-quality, structured lesson content that combines text, visuals, and examples, optimized for comprehension and accessibility.

Preconditions: Access to well-curated, tagged curriculum data and media assets is essential for relevant lesson generation.

Postconditions: Generated lessons are cached for rapid delivery, ensuring smooth learner interaction and consistent quality.

Dependencies: Strong integration with the content database (PostgreSQL) and media storage (MinIO) to leverage rich educational resources.

SLA: Guarantees responsive lesson generation, typically within one second, to maintain learner engagement.

Failure Handling: In case of generation issues, defaults gracefully to cached lessons or placeholders, preserving the learning flow.

6.3.3. Quiz & Challenge Agent Contract

Inputs: Comprehensive learner mastery profiles, topic details, and historical quiz data guide adaptive quiz construction.

Outputs: Adaptive quiz items with difficulty calibrations and misconception-focused distractors, accompanied by transparent rationale explaining adaptive decisions.

Preconditions: Requires accurate, up-to-date learner mastery information to tailor question difficulty effectively.

Postconditions: Updates mastery tracking with each quiz attempt, logging difficulty adjustments to inform future adaptations.

Dependencies: Utilizes LLM services for generating nuanced distractors and accesses learner and content databases to align assessments.

SLA: Delivers adaptive quizzes promptly, responding within 500 milliseconds for most requests to maintain engagement.

Failure Handling: On LLM or data access failure, falls back to trusted static question banks, ensuring uninterrupted assessment.

6.3.4. Feedback & Motivation Agent Contract

Inputs: Receives detailed hint requests inclusive of question context, learner response history, and identified misconceptions.

Outputs: Context-aware, tiered hints and motivational messages that guide learners toward mastery without frustration, plus triggers for re-quizzes when reinforcement is needed.

Preconditions: Requires sufficient learner interaction data, including error counts and hint usage history, to tailor feedback effectively.

Postconditions: Feedback is logged for analytics and used to adapt future interactions, promoting a personalized, nurturing learning environment.

Dependencies: Combines scripted feedback templates from PostgreSQL with dynamic LLM-generated hints, maintaining safety and relevance through guardrails.

SLA: Ensures timely hint delivery within 1.5 seconds to maintain learner flow and reduce frustration.

Failure Handling: When LLM services are unavailable, it seamlessly reverts to scripted hints, with fallback notifications sent to the Planner.

6.3.5. Analytics & Attention Agent Contract

Inputs: Real-time telemetry encompassing response times, error rates, and inactivity periods, reflecting learner focus and performance.

Outputs: Continuously updated mastery and attention metrics, drift detection alerts, and pacing recommendations that feed adaptive content sequencing.

Preconditions: Requires uninterrupted access to timely telemetry streams to accurately assess learner states.

Postconditions: Persists updated metrics and alerts, providing actionable data to the Planner and dashboards.

Dependencies: Relies on robust integration with event and learner databases, Redis caching for performance, and analytic algorithms for real-time processing.

SLA: Processes attention analytics within one second, ensuring timely intervention to maintain engagement.

Failure Handling: In cases of telemetry gaps, temporarily suspends drift detection and relies on last known metrics, prioritizing system stability.

6.4. Privacy Guardrails

LUMO is designed with strong privacy and security principles to protect the sensitive information of its young learners and comply with relevant educational data regulations.

6.4.1. Data Minimization and Consent

LUMO collects only the data necessary for personalized learning, avoiding excessive personally identifiable information (PII) beyond essential identifiers like email and username. The platform does not track student location or device fingerprints and collects biometric data only with explicit user consent. All data collection is transparent, with clear privacy notices and an opt-in model for advanced features such as attention tracking.

6.4.2. Data Retention and Anonymization

User event data is retained for a default period of 90 days. After this period, data is anonymized by removing all user-identifying information, ensuring privacy while preserving aggregated data for research and analytics. Users and parents retain the right to request data deletion at any time. This retention policy is enforced automatically by database-level functions that cleanse data post-retention.

6.4.3. Compliance and Incident Response

LUMO maintains strict adherence to educational privacy regulations, including FERPA, COPPA, and GDPR. The platform enforces parental consent workflows, limits on data sharing, and mechanisms for users to exercise data rights. In case of data breaches or privacy violations, automated detection, containment, notification, and remediation protocols are in place, complemented by regular privacy audits, penetration testing, and user feedback mechanisms to continuously improve privacy posture.

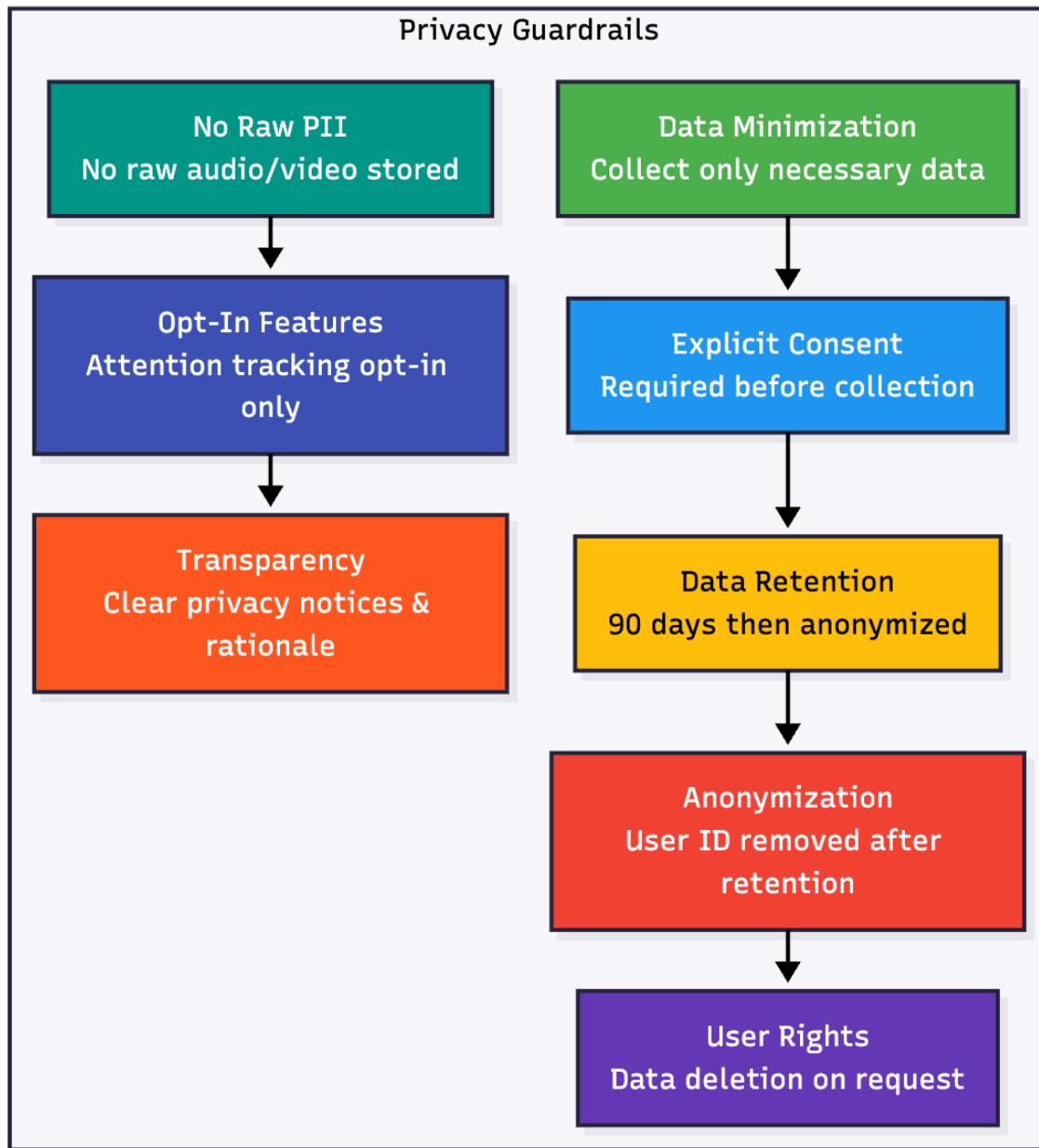


Figure 11. Privacy Guardrails

6.5. Security Controls

6.5.1. Role-Based Access Control and Authentication

Access to data is strictly controlled via Role-Based Access Control (RBAC). Students can only view their own data; parents have access limited to their children's data; teachers can see data for their assigned students; and administrators have system-wide,

audit-logged access. Authentication is handled securely through JWT-based tokens with short lifetimes, ensuring session security.

6.5.2. Data Encryption and Audit Logging

All data is encrypted both at rest (AES-256) and in transit (TLS 1.3). Detailed audit logs track data access and changes to ensure compliance with FERPA, COPPA, and GDPR regulations. The system includes rate limiting and permission validations on all API endpoints to prevent unauthorized data harvesting or bulk exports.

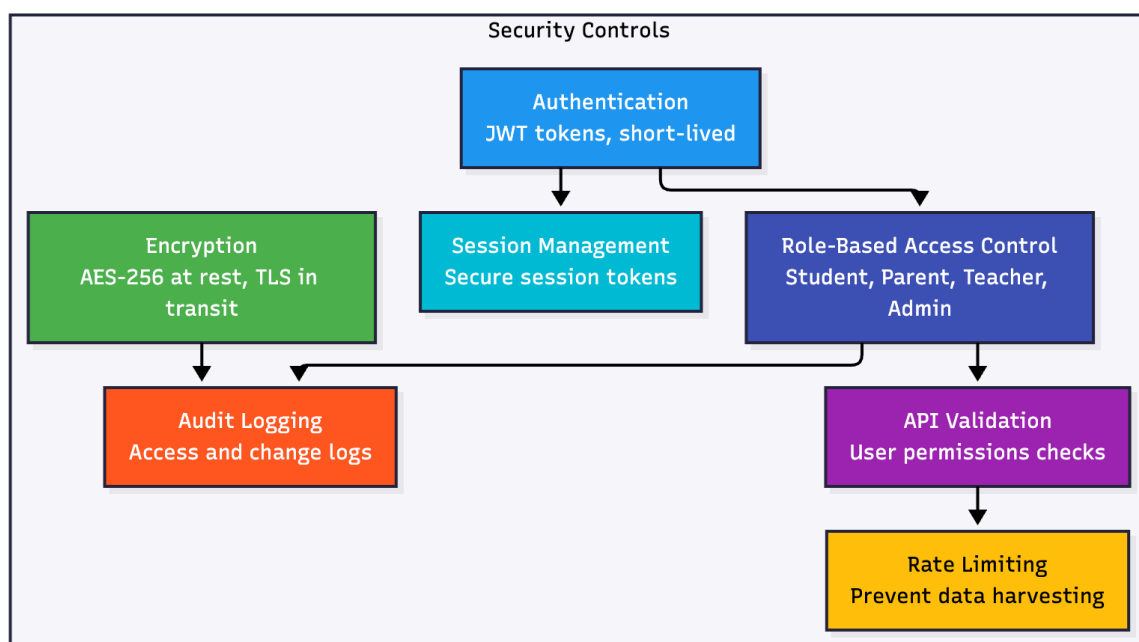


Figure 12. Security Controls

Chapter 7. QA, Performance, Deployment Plan

LUMO's QA and deployment strategy ensures functional correctness, ethical safety, and operational scalability. The plan bridges the gap between prototype testing and production readiness through layered validation, observability, and progressive rollout.

Table 3 – LUMO's QA, Performance and Deployment Practices over two phases

Dimension	Phase 0 - Prototype / Research - Late Dec - Early Feb	Phase 1 - Scaling / Pilot - Feb to May
QA Focus	Manual testing, traceability draft	Automated coverage metrics
LLM Review	Human rubric	Hybrid human + automated filters
Performance Goal	Collect baseline observations	Define response-time budgets and SLOs
Deployment	Local	Docker setup
Observability	Log inspection	Dashboards and alerts
Ethical Review	Ad-hoc consent forms	Formal parental consent workflow

7.1 QA Strategy and Traceability

LUMO's QA framework adopts a layered test-pyramid strategy (Fig. 7.1) suitable for multi-agent systems. Contract tests are used to confirm message schemas and data exchange among agents. Component tests verify each agent's internal logic (Lesson, Quiz, Feedback, Attention). To ensure orchestrated learning loops behave coherently, we will have Integration / End-to-End tests. The Exploratory and non-functional tests to evaluate usability, ethical safeguards, and early stability.

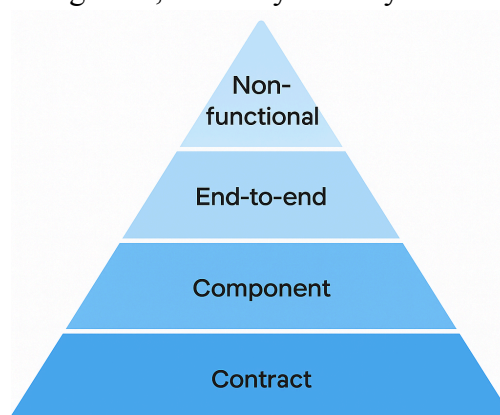


Figure 13. Test Pyramid for LUMO QA

To promote transparency, reproducibility, and future scalability of the test suite, even in the prototype stage, a verification is planned for functional and non functional requirements.

Privacy compliance (COPPA / GDPR) is a core ethical requirement for LUMO, as it handles sensitive data from children. Personally Identifiable Information (PII) refers to any data that can directly or indirectly identify an individual. For LUMO, this includes names, email addresses, and phone numbers. Since the Attention Agent may process multimedia inputs, photos, voice recordings, and video frames are also considered PII, along with any other data elements that could reveal a student's identity.

Table 4 – Requirement Traceability Matrix

Requirement ID	Description	Verification Approach	Expected Evidence
FR-Lesson-Design	Generate curriculum-aligned micro-lessons	Manual review + automated format validation	Logged examples conforming to schema
FR-Quiz-Adapt	Adjust difficulty based on performance	Simulated answer patterns	Logs show difficulty shift and rationale
FR-Feedback-Hints	Provide tiered hints	Interactive session test	Sequential hints retrieved correctly
NFR-Transparency	Explain adaptive decisions	UI inspection + API call check	Response includes rationale text
NFR-Privacy	Protect learner identity	Prompt inspection	No PII tokens in payload

The team must manually review sample lessons to verify that the generated content is curriculum-aligned, age-appropriate, and contextually accurate. In parallel, automated validators will ensure that each lesson output conforms to the defined JSON schema (e.g., title, learning objective, example, image link). This dual validation process ensures that every lesson is both educationally sound and technically well-structured.

To test adaptive difficulty logic, student responses are simulated. These will have mixed responses of some correct, some incorrect, and mixed. With this we observe whether quiz difficulty rises or falls appropriately. The Planner and Quiz Agent logs will then be analyzed to confirm that difficulty adjustments occur within the expected thresholds and that the rationale for each adaptation is recorded.

Interactive session test is a hands-on functional test. When one opens the lesson or quiz interface, trigger wrong answers, and check whether the three-level hint ladder, light cue → guided step → worked example, appears in proper sequence and that transitions between levels occur smoothly. The session logs will be inspected to ensure hints are contextually relevant and supportive in tone.

Every adaptive action such as difficulty change, hint trigger, feedback, must include a visible “why” explanation. For the API call check, the user interface and inspect backend API responses are reviewed to confirm that the “rationale” field is always present and displayed in the dashboard or feedback text. This confirms that parents and learners can understand the system’s reasoning.

For the prompt inspection, before any data is sent to the LLM, the payloads are inspected to verify that no personally identifiable information (PII) is included. Synthetic user identifiers (e.g., learner #123) will be used instead. Logs will be reviewed to ensure PII remains fully redacted throughout the workflow.

7.2 Usability and Human-Centric Validation

Given LUMO's target audience of elementary students and its reliance on the Parent/Teacher Dashboard, we must explicitly validate the system's Usability and Child-Friendly Interface (as mandated in the Non-Functional Requirements).

Table 5 - Human centric validation of components

Component Tested	Methodology	Key Metrics / Validation Points
Student Interface (LUMO Client)	Think-Aloud Protocol and Task Completion Rate	Child-Friendly: Assess ease of navigation, clarity of instructions, and emotional response to the interface. Engagement: Measure Time-on-Task and adherence to suggested breaks.

Parent Dashboard	System Usability Scale (SUS) and Scenario-Based Interviews	Actionability: Validate that the attention and mastery reports are clear, understandable, and enable immediate decision-making (e.g., "Can a teacher quickly identify the 3 students most likely to disengage?").
Teacher Dashboard (Stretch Goal)	System Usability Scale (SUS) and Scenario-Based Interviews	Actionability: Validate that teachers can quickly identify at-risk students and take appropriate action based on mastery and attention data.

7.2 LLM Output and Ethical Quality Assurance

Keeping in mind that LUMO is catered to elementary students, the app should deliver age-appropriate content with the right tone and safety. This is on top of the quality checks for LUMO which uses generative models to produce educational explanations.

Table 6 - Ethical issues to be taken care of in LUMO

Dimension	Purpose	Evaluation Method
Age-appropriateness	Ensure vocabulary and context suit elementary learners	Human review; readability index
Correctness and Clarity	Validate factual accuracy and reasoning	Expert sampling; peer comparison
Tone and Motivation	Maintain encouraging, inclusive tone	Sentiment analysis; human rubric

Ethical and Safety Compliance	Prevent biased or unsafe responses	Red-team prompting; guardrail testing
--------------------------------------	------------------------------------	---------------------------------------

7.2.1 Red-Team and Fallback Procedure

A red-team is an adversarial testing group/process that deliberately probes a system for weaknesses by simulating realistic attacks, misuse, or failure modes. It checks for malicious prompts, clever jailbreaks, data-leaks, adversarial inputs, privacy violations, and social/ethical harms.

Agentic systems add complexity that expands the attack surface. Multiple agents exchange messages, there could be protocol/contract attacks. While an agent that calls external tools/APIs can have a tool-invocation risk. LLMs can be prompted indirectly via user content or other agents leading to prompt-injection. Multi-turn reasoning creates opportunities for drift, looped failures, or compounding hallucinations. There are safety/privacy stakes with minors LLM output tone, PII leakage. So red-teaming finds composite failure modes that unit tests, or even plain E2E tests, often miss.

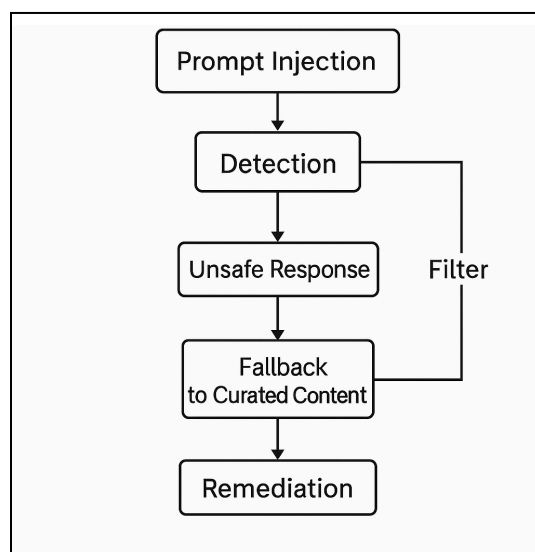


Figure 14. Detection of prompt injection and automatic fallback to safe remediation.

Implementation Steps:

- Define threat model & scope : Identify potential attackers, their objectives, and protected assets (e.g., children’s privacy, curriculum integrity).
- Seeded prompt set: Create adversarial prompts grouped by category - injection, toxicity, privacy bait, social engineering.
- Triage & label: Evaluate model outputs as Safe / Unsafe / Ambiguous and record their cause (prompt, agent, tool, chain-of-thought).
- Implement simple guards: Apply prefix sanitizers, deny-lists, and automatic fallback to curated content for critical failures.

- Produce a short report with findings + regression tests (golden adversarial cases).

7.3 Performance Planning

LUMO's performance plan aims to observe patterns. Our emphasis is on collecting baseline indicators under simulated classroom loads.

Table 7 - Metrics considered to test performance for LUMO

Metric	Test Method
Planner latency	Planner latency
Quiz Agent response	Quiz Agent response
Lesson render time	Lesson render time
LLM fallback rate	LLM fallback rate
Dashboard update delay	Dashboard update delay
Concurrent users	Concurrent users

Initial goals:

- Validate that event logs conform to schema definitions.
- Verify temporal coherence (lesson → quiz → feedback order).
- Ensure privacy redaction before storage.
- Generate pilot dashboards visualizing retry counts, attention drift, and mastery gain.

7.3.1 Automated Evals for Agents

Each LUMO agent undergoes automated evaluations (evals) that benchmark accuracy, latency, and ethical compliance. These evals form part of the QA pipeline and are executed regularly to detect regressions or unsafe behaviors. For instance, the Quiz Agent's evals measure adaptive difficulty accuracy, while the Feedback Agent's evals check for motivational tone and factual correctness.

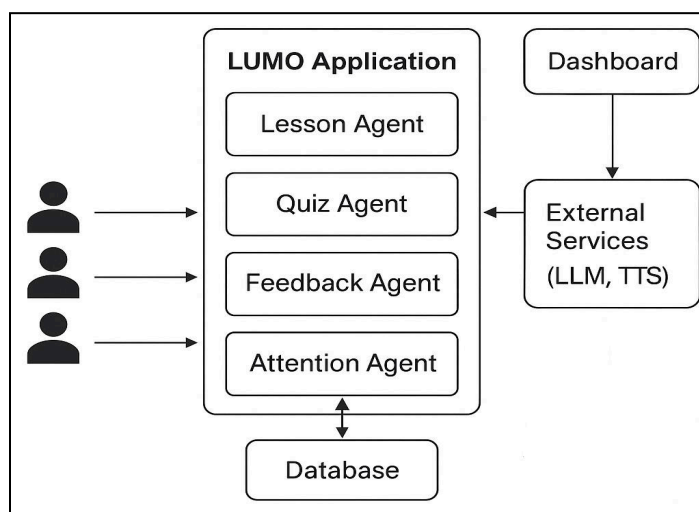


Figure 15. Conceptual Load Flow for Multi-Agent Interaction

Figure 15 shows a conceptual flowchart illustrating how student interactions traverse the LUMO system during a typical session. Multiple learners submit requests that the Planner sequences and dispatches to specialized agents. Agents may consult the persistent database and external services such as LLM providers before returning outputs that the Planner composes into the Dashboard view.

Data quality governs how attention metrics, mastery updates, and feedback traces are captured.

7.3.2 Advanced Agentic System Tests

LUMO incorporates specialized testing practices designed for agentic systems, where multiple agents collaborate dynamically to achieve educational goals. These advanced tests ensure that orchestration, data flow, and decision-making remain reliable, ethical, and pedagogically consistent across all stages of deployment.

1. **Inter-Agent Protocol Tests:** Verify that all agents communicate according to defined contracts and message schemas. These tests validate that inputs, outputs, and rationales exchanged between the Planner, Lesson, Quiz, Feedback, and Attention agents remain consistent and traceable.

2. **Orchestration Consistency Tests:** Ensure that the Planner correctly sequences agent interactions and gracefully handles agent unavailability. Tests simulate agent failures and confirm that fallback logic is activated as intended.
3. **State Synchronization and Memory Consistency Tests:** Confirm that shared learner data, including mastery scores and attention metrics, stay synchronized across agents. These tests detect race conditions, delayed updates, or data drift between the Feedback and Analytics agents.
4. **Goal Adherence and Policy Compliance Tests:** Evaluate whether the Planner’s task sequencing aligns with pedagogical and ethical rules. For example, the Planner should not assign difficult quizzes immediately after detecting low attention or repeated errors.
5. **Regression and Drift Evals:** Continuously compare new model outputs against baseline “golden” responses to detect degradation in correctness, tone, or engagement quality. Drift evals help identify when retraining or prompt revision is required.
6. **Safety and Sandbox Isolation Tests:** Validate that each agent operates within its designated sandbox, preventing unauthorized data access or external API misuse. These tests confirm compliance with privacy guardrails and system security policies.
7. **Scalability and Resource Utilization Tests:** Assess the performance of multi-agent interactions under increasing workloads. Load simulations measure concurrency tolerance, CPU/memory usage, and autoscaling efficiency to ensure stable operation during classroom-scale deployments.

Given LUMO’s complexity, to validate adaptive logic and agent coordination under realistic conditions, a Scenario-Based Testing and A/B Path Tracing will be used. For instance, a student answers multiple questions incorrectly and the Attention Agent reports low focus. Then the Quiz Agent should flag the pattern, prompting the Planner to skip the next quiz and trigger a Feedback Agent intervention such as a break suggestion (“Time for a 2-minute stretch!”). Complementing this, A/B Path Testing compares different mastery model update paths, such as Deep Knowledge Tracing (DKT) versus Bayesian Knowledge Tracing (BKT), to empirically confirm that LUMO’s DKT-based approach more accurately predicts learner mastery and guides the Lesson Designer Agent toward more effective lesson sequencing.

7.4 Deployment Approach

LUMO will adopt a two-stage environment ladder. At each stage a QA checklist, ethical and privacy review and integration tests must be completed before progressing. If any test or audit fails, fallback actions include disabling LLM routes or reverting to scripted lessons.

Table 8 - Lumo's technology stack used at each stage

Stage	Description	Infrastructure
Local Dev	Docker Compose for Next.js (3000), FastAPI (8000), PostgreSQL (5432), Redis (6379), MinIO (9000/9001). Used for feature development and schema testing.	Developer machines (VS Code + Docker Desktop)
Production	Scaled multi-instance FastAPI behind Load Balancer; Next.js served via Vercel/CloudFront; PostgreSQL Primary + Replica; Redis Cluster; MinIO S3; Gemini API + Monitoring.	Cloud managed

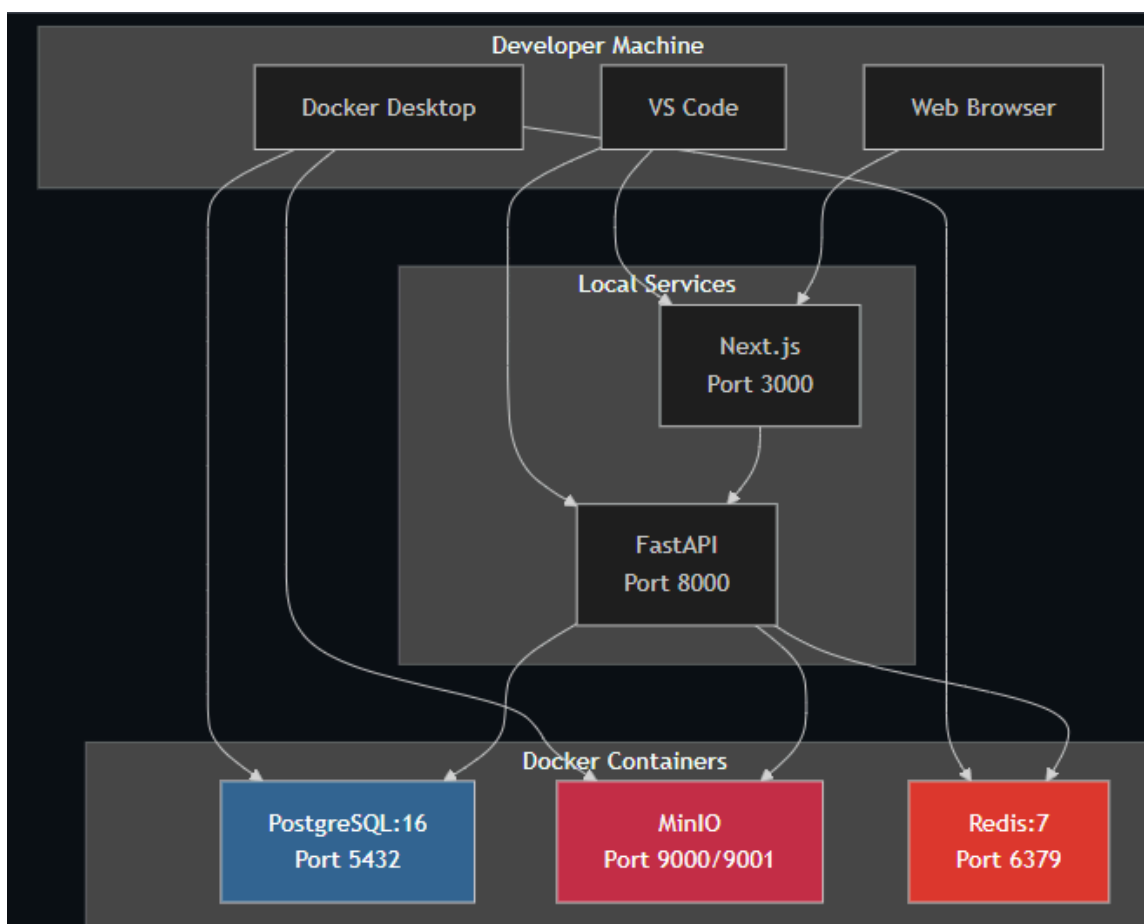


Figure 16. Deployment diagram for LUMO

7.5 Risk Awareness and Mitigation

A concise register tracks early technical and ethical uncertainties.

Table 9 – Preliminary Risk Matrix

Risk	Impact	Mitigation
Privacy concerns from guardians	High	Default “no-sensor” mode; opt-in analytics
Model latency or cost spikes	Medium	Local cache; deferred processing
Bias or inappropriate tone	High	Human-in-the-loop review; continuous red-team tests
Limited content coverage	Medium	Expand OER corpus; educator tagging

7.6 Production Monitoring and Feedback

Following deployment, a continuous feedback loop will be maintained through real-time dashboards that monitor system health and usage. Automatic alerts will notify the team of LLM errors, latency spikes, or privacy anomalies. In addition, monthly review meetings will be conducted to refine guardrails, update curriculum coverage, and adjust performance budgets based on observed trends.

Figure 17 illustrates the live monitoring setup for LUMO's production environment, showing integration between FastAPI logs, Redis queues, and Datadog alerts.

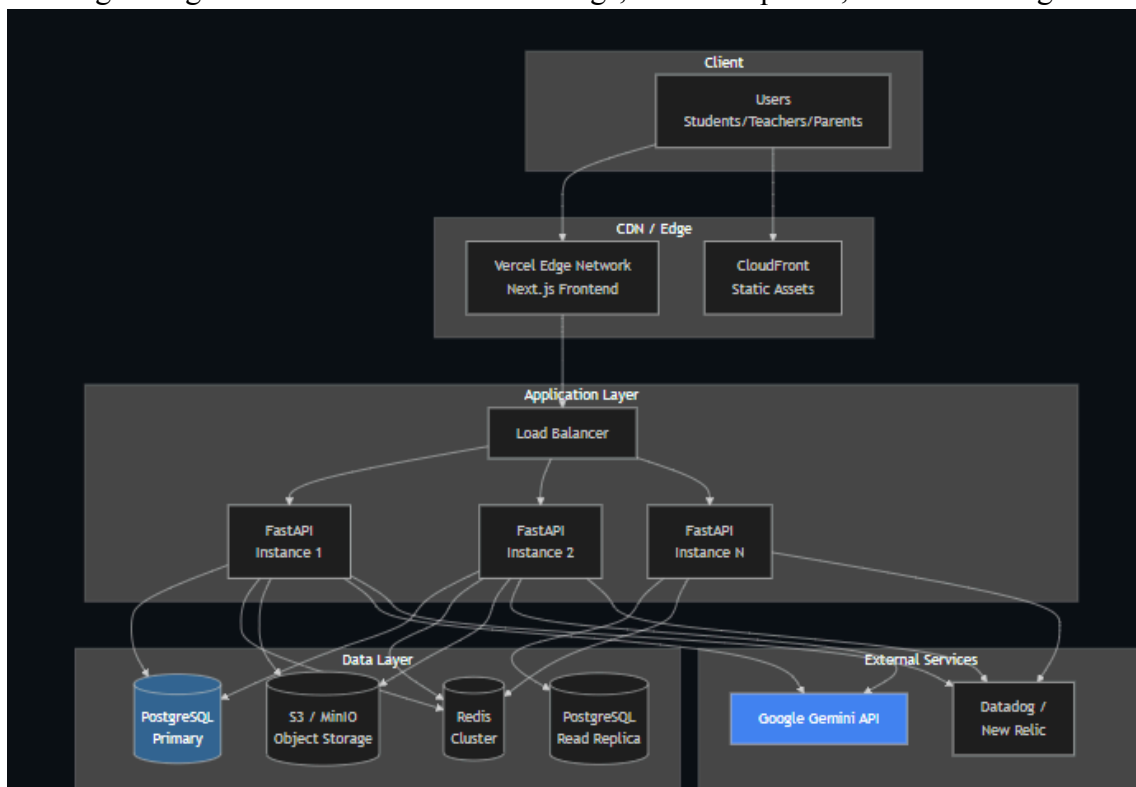


Figure 17. Production diagram for LUMO

Chapter 8. Implementation Plan and Progress

The implementation plan for this semester would be to combine research and development to reproduce credible baselines, prototype contributions that push the current SOTA techniques and deliver a demo and technical report by the end of the semester. The work would continue in the next semester with scaled evaluations and deeper modelling and fine tuning.

8.1 Programming and Execution Environment

The first and foremost process would be to set up the environment within our personal devices. The team would be using VSCode on either MacOS/Linux/Windows. To maintain version control we have agreed to use Github repo with three layers or branches, one for the web app consisting of the relevant Next.js/Typescript code. This would be used to maintain the code for the lesson, quiz, feedback interfaces and the dashboard views. The next layer would be a FastAPI service exposing endpoints for content, sessions and analytics. Finally there would be an experimental layer to reproduce current SOTA baseline and for simulations. Environments would be reproducible via *uv*, a python package and project manager and *npm* for the web stack. The team has decided to use a docker compose file to bring up the database for storing analytics results, lesson content locally, allowing the entire stack to run without external services.²

8.2 Development Tools for the Team Members

Table 3. List of respective development tools

Member	Component	Core Stack	LLM use	Data storage
Gautam	Lesson Design and Generation	React and Next.js (Typescript), MDX authoring, Figma, Storybook	Provides tagged content (e.g., misconception types) to contextualize LLM prompts for other	Postgres (Content Schema)

²8. Visual Studio Code, *Code Editing. Redefined.*, available at: <https://code.visualstudio.com/>

9. GitHub, *Where the world builds software*, available at: <https://github.com/>

10. uv, *Python package and project manager*, available at: <https://github.com/astral-sh/uv>

11. npm, *Node package manager*, available at: <https://www.npmjs.com/>

12. Docker Compose, *Define and run multi-container applications*, available at: <https://docs.docker.com/compose/>

13. MDX, *Markdown for the component era*, available at: <https://mdxjs.com/>

14. Figma, *Collaborative interface design tool*, available at: <https://www.figma.com/>

15. Storybook, *Frontend workshop for building UI components*, available at: <https://storybook.js.org/>

			components.	
Bhavya	Feedback and Motivation	React (Next.js), FastAPI/Python	Generates tiered, Socratic hints for users based on their specific misconceptions.	Interacts with Redis (task queue) and Postgres for storing/retrieving results.
Alshama	Quiz Generation	FastAPI/Python, Celery	Generates plausible incorrect answers (distractors) for quiz questions.	Postgres (Content Schema)
Nivedita	Analytics and Attention	Python, Postgres	Uses the outputs of user interactions (influenced by LLM feedback) to update the learner model.	Postgres (Events Schema & Learner Model Schema)

8.3 Implementation Flow

8.3.1 Phase 1: Environment and Contracts

- **Gautam:** Sets up GitHub repo and Docker environment and finalizes event schemas.
- **Bhavya:** Collaborates on Feedback API contracts and privacy guardrails.
- **Alshama:** Provides input on quiz API contracts and heuristics.
- **Nivedita:** Defines event ingestion and telemetry contracts for analytics.

8.3.2 Phase 2: Baseline Reproduction

- **Gautam:** Builds minimal lesson renderer with accessibility checks.
- **Bhavya:** Develops single-level hint loop and feedback placeholder; wires re-quiz trigger.
- **Alshama:** Implements baseline quiz heuristics and API stub with Postgres.

- **Nivedita:** Enables event ingestion pipeline; computes latency metrics; validates dashboards.

8.3.3 Phase 3: Vertical Workflow for Lesson to Quiz

- **Gautam:** Prepares micro lesson packs; finalizes lesson-to-quiz contracts; integrates lesson renderer with quiz engine.
- **Bhavya:** Connects feedback placeholders to capture errors and trigger hints.
- **Alshama:** Exposes quiz endpoints and finalizes single-level quiz workflow.
- **Nivedita:** Stabilizes logging and enhances telemetry.

8.3.4 Phase 4: Closed Loop with Feedback and Attention Hooks

- **Gautam:** Adds example selection policies and content strings.
- **Bhavya:** Integrates adaptive tiered hints and LLM-drafted hints under guardrails; measures retries-to-mastery.
- **Alshama:** Implements LLM-assisted summaries and enhances quiz features.
- **Nivedita:** Delivers initial attention score and basic break scheduler; integrates attention hooks.

8.3.5 Phase 5: Analytics, Privacy and Transparency

- **Gautam:** Expands sequencing rules and rationale coverage.
- **Bhavya:** Adds motivational nudges to feedback workflow.
- **Alshama:** Implements timed challenges and partial-credit scoring; performs QA.
- **Nivedita:** Publishes dashboards, materializes metrics, completes privacy reviews.

8.3.6 Phase 6: Verification and Validation

- **Gautam:** Stabilizes lesson renderer; conducts usability testing; curates demo content.
- **Bhavya:** Triage UI bugs; scripts walkthroughs and user guides.
- **Alshama:** Benchmarks latency; finalizes quiz exports; runs load tests.
- **Nivedita:** Runs load/stress tests on analytics; synthesizes final reports.

8.4 Progress

Table 4. Progress for LUMO

Work Area	Research & Contributions
-----------	--------------------------

Lesson Designer Agent – Gautam Santhanu Thampy	Conducted detailed research on mastery-based learning, curriculum personalization, and adaptive lesson sequencing using Deep Knowledge Tracing (DKT) models. Designed the initial structure for the Lesson Designer Agent, including lesson generation flow, tagging logic, and integration with Planner.
Quiz & Challenge Agent – Alshama Mony Sheena	Researched LLM-based question generation, trap-based distractors, and adaptive difficulty adjustment methods. Designed the Quiz Agent architecture and API contracts for integration with the Lesson and Feedback Agents.
Feedback & Motivation Agent – Bhavya Jain	Studied literature on affective feedback systems, motivational learning psychology, and Socratic hint frameworks. Designed feedback flow logic, tone guardrails, and ethical motivation triggers.
Analytics & Attention Agent – Nivedita M Nair	Performed research on attention measurement (CPT, ACS, EEG, gaze tracking) and behavioral modeling. Designed the attention agent, outlining data models, scoring logic, APIs, and ethical guardrails.
Architecture & Integration (Team Collaboration)	Researched multi-agent system design and finalized the Planner-based orchestration architecture. Defined inter-agent contracts, data flow diagrams, and shared schemas (PostgreSQL + Redis). Created github environment setup for development.
Testing & Quality Assurance (Team Collaboration)	Designed the testing strategy, including unit, integration, and ethical QA plans. Drafted requirement traceability and red-team validation structure for future implementation.

Project Planning & Tracking	Transitioned from Monday.com to a shared Excel tracker for version control and offline coordination. All milestones, responsibilities, and research summaries documented for team visibility.
-----------------------------	---

Chapter 9. Project Schedule

The project schedule lays out LUMO's plan for Fall 2025 across 5 sprints leaving room for buffer for scope creep. The schedule caters to continuous utilization of all the team members with minimal overlap. Milestones are needed at the end of every sprint to ensure that the project is on track.

The complete Gantt chart and task board can be accessed here: [Project schedule link](#)

Figure 19. View of LUMO's progress in Excel